- A style classification head for dynamic style inference.
- A StyleAdapter module that modulates the hidden states of a transformer-based language model.

This approach allows for fine-grained stylistic control without significantly altering the base language model architecture. By incorporating style adaptation after every transformer layer, we create stronger style-specific representations throughout the model, enhancing both style awareness and consistency.

To verify the effectiveness of our approach, we conducted extensive experiments on multiple datasets, including Shakespeare's works (shakespeare_char), enwik8, and text8. Our results demonstrate that the Multi-Style Adapter achieves high style consistency while maintaining competitive language modeling performance. Key findings include:

- Improved validation losses compared to the baseline model, with the best performances on enwik8 (0.9488) and text8 (0.9145).
- Near-perfect style consistency scores across all datasets (0.9667 for shakespeare_char, 1.0 for enwik8 and text8).
- A trade-off in computational efficiency, with inference speeds of approximately 400 tokens per second compared to 670 in the baseline.

The main contributions of this paper are:

- A novel Multi-Style Adapter architecture that enhances style awareness and consistency in character-level language models.
- An effective method for balancing style adaptation and language modeling capabilities within a single model.
- Comprehensive experiments demonstrating improved validation losses and high style consistency across multiple datasets.
- Analysis and visualization of learned style embeddings and style-specific attention patterns, providing insights into the model's style representation capabilities.

In the following sections, we discuss related work, provide background on language models and style adaptation, detail our method, describe our experimental setup, present our results, and conclude with a discussion of the implications and future directions for style-aware language models.

Future work could focus on optimizing the computational efficiency of the Multi-Style Adapter, exploring more sophisticated style representation techniques, and investigating the model's performance on style transfer tasks and its ability to generalize to unseen styles.

2 Related Work

The field of style-aware language models has seen significant advancements in recent years, with researchers exploring various approaches to incorporate and control stylistic elements in text generation. Our Multi-Style Adapter builds upon these foundations while addressing some limitations of existing approaches.

Shen et al. (2017) proposed a method for style transfer without parallel data, using cross-alignment to separate content from style. While this approach laid the foundation for many subsequent studies in style-aware language modeling, it primarily focuses on transferring between two distinct styles. In contrast, our Multi-Style Adapter learns multiple style representations simultaneously, allowing for more flexible style generation and adaptation.

Pfeiffer et al. (2020) introduced AdapterFusion, a method for combining multiple adapters in language models, which allows for non-destructive task composition and transfer learning. This approach is conceptually similar to our Multi-Style Adapter, as both use adapter modules to specialize the base model for different tasks or styles. However, our method differs in its integration of style embeddings and a style classification head, which allows for dynamic style inference and adaptation during both training and inference.

CAUTION!!! THIS PAPER WAS AUTOMOUSLY GENERATED BY THE AI SCIENTIST

The CTRL model Keskar et al. (2019) demonstrates the ability to generate text conditioned on specific control codes, offering a different approach to style-aware language modeling. While CTRL's use of control codes shares similarities with our Multi-Style Adapter's use of style embeddings, our approach focuses on learning and adapting to styles during training rather than using predefined control codes. This allows our model to potentially discover and utilize more nuanced style representations that may not be captured by predefined categories.

Our Multi-Style Adapter addresses several limitations of these existing approaches:

1. Flexibility: Unlike methods that rely on predefined style categories or control codes, our approach learns style representations during training, allowing for more flexible and adaptable style modeling.

2. Granularity: By incorporating style adaptation after every transformer layer, we create stronger style-specific representations throughout the model, enhancing both style awareness and consistency.

3. Scalability: Our approach can handle multiple styles within a single model, making it more scalable than methods that require separate models or extensive fine-tuning for each style.

4. Dynamic Adaptation: The style classification head allows our model to dynamically infer and adapt to styles during inference, even for unseen text.

The experimental results presented in this paper demonstrate the effectiveness of our approach. Across multiple datasets (shakespeare_char, enwik8, and text8), we achieve high style consistency scores (0.9667 for shakespeare_char, 1.0 for enwik8 and text8) while maintaining competitive language modeling performance. These results suggest that our Multi-Style Adapter effectively balances style adaptation and language modeling capabilities, addressing a key challenge in style-aware language generation.

In conclusion, while existing work has made significant strides in style-aware language modeling, our Multi-Style Adapter offers a novel approach that combines the strengths of adapter-based methods with learned style representations. This combination allows for more flexible and consistent style-aware text generation, as demonstrated by our experimental results.

3 Background

The development of style-aware language models builds upon several key advancements in natural language processing and deep learning. This section provides an overview of the foundational concepts and prior work necessary for understanding our Multi-Style Adapter approach.

3.1 Language Models and Transformers

Language models have evolved from simple n-gram models to sophisticated neural network-based architectures Goodfellow et al. (2016). A pivotal breakthrough came with the introduction of the Transformer architecture Vaswani et al. (2017), which revolutionized the field due to its ability to capture long-range dependencies and process input sequences in parallel. The Transformer's self-attention mechanism allows the model to focus on relevant parts of the input when generating each output token, greatly enhancing its ability to capture context and produce coherent text Bahdanau et al. (2014).

Building upon the Transformer architecture, the Generative Pre-trained Transformer (GPT) family of models has further advanced language generation capabilities Radford et al. (2019). These models, trained on vast amounts of text data, have demonstrated remarkable proficiency in generating coherent and contextually appropriate text across various domains and tasks.

3.2 Style Adaptation in Language Models

While language models have made significant strides in generating fluent text, controlling the style of the generated content remains a challenge. Style adaptation in language models aims to enable the generation of text that adheres to specific stylistic characteristics while maintaining coherence and fluency. This capability is crucial for applications ranging from creative writing assistance to personalized content generation.

AUTONOMOUSLY GENERATED BY THE AI SCIENTIST
THIS PAPER WAS

Previous approaches to style-aware language modeling include:

- Fine-tuning pre-trained models on style-specific datasets
- Incorporating style tokens or embeddings as additional input
- Using conditional language models with style as a conditioning factor

Our Multi-Style Adapter builds upon these ideas, introducing a more flexible and adaptive approach to style-aware language generation.

3.3 Problem Setting

In this work, we address the task of style-aware language modeling. Given a sequence of input tokens $x = (x_1, \ldots, x_T)$ and a desired style s, our goal is to generate a sequence of output tokens $y = (y_1, \ldots, y_N)$ that not only continues the input sequence coherently but also adheres to the specified style. Formally, we aim to model the conditional probability distribution:

$$P(y|x, s) = \prod_{t=1}^{N} P(y_t|y_{<t}, x, s) \tag{1}$$

where $y_{<t}$ represents all tokens generated before y_t.

To incorporate style awareness, we introduce a set of learnable style embeddings $E_s \in \mathbb{R}^{K \times D}$, where K is the number of predefined styles and D is the embedding dimension. These style embeddings are used to modulate the hidden states of the language model, allowing for style-specific text generation.

Our approach makes the following assumptions:

- The set of styles is predefined and finite.
- The style of the input sequence is not explicitly provided and must be inferred by the model.
- The model should be capable of maintaining style consistency throughout the generated sequence.

By extending the GPT architecture with our Multi-Style Adapter, we aim to enhance style awareness and consistency in character-level language generation while maintaining competitive language modeling performance.

4 Method

Building upon the problem formulation introduced in Section 3, we present our Multi-Style Adapter approach to enhance style awareness and consistency in character-level language models. Our method extends the GPT architecture by introducing three key components: learnable style embeddings, a style classification head, and a StyleAdapter module.

4.1 Learnable Style Embeddings

We define a set of learnable style embeddings $E_s \in \mathbb{R}^{K \times D}$, where $K = 4$ is the number of predefined styles and $D = 64$ is the embedding dimension. These embeddings serve as compact representations of different writing styles:

$$E_s = [e_1, e_2, \ldots, e_K], \quad e_i \in \mathbb{R}^D \tag{2}$$

The style embeddings are initialized randomly and updated through backpropagation during training, allowing the model to discover and refine style representations that are most useful for the task at hand.

CAUTION!!! THIS PAPER WAS AUTONOMOUSLY GENERATED BY THE AI SCIENTIST